Intelligent Agents

Aim:

To demonstrate a simple reflex agent — a vacuum cleaner agent that perceives its location and cleanliness status, and acts using condition-action rules, without maintaining internal state history.

Theory:

An intelligent agent perceives its environment through sensors and acts upon it through actuators, aiming to maximize a performance measure. A simple reflex agent is the most basic agent type: it selects actions based solely on the current percept, ignoring percept history, using condition-action rules of the form "if condition then action." Here, the environment is a two-location world (A and B), each of which can be Dirty or Clean.

The agent's rule set is:

- If current square is dirty → Clean it.
- If square is clean and agent is at A → move Right.
- If square is clean and agent is at B → move Left.

This models the classic AIMA (Russell & Norvig) vacuum-world example, illustrating the perceive → decide → act cycle central to agent architectures.

Code:

```
"""
Intelligent Agent Demonstration - Simple Reflex Vacuum Cleaner Agent
"""


class SimpleEnvironment:
    """A 1D environment with two locations: A and B, each can be Clean or Dirty."""
    def __init__(self):
        self.status = {'A': 'Dirty', 'B': 'Dirty'}

    def is_dirty(self, location):
        return self.status[location] == 'Dirty'

    def clean_location(self, location):
        self.status[location] = 'Clean'
```

```
    def display(self):
        return dict(self.status)


class ReflexVacuumAgent:
    """A simple reflex agent that decides actions based only on current percept."""
    def __init__(self, environment):
        self.environment = environment
        self.location = 'A'
        self.performance = 0  # counts number of clean actions (performance measure)

    def perceive(self):
        # Percept = (current location, status of current location)
        return self.location, self.environment.status[self.location]

    def decide_action(self, percept):
        location, status = percept
        if status == 'Dirty':
            return 'Clean'
        elif location == 'A':
            return 'Right'
        else:
            return 'Left'

    def act(self, action):
        if action == 'Clean':
            self.environment.clean_location(self.location)
            self.performance += 1
        elif action == 'Right':
```

```
            self.location = 'B'
        elif action == 'Left':
            self.location = 'A'

    def run(self, steps=5):
        print(f"{'Step':<6}{'Location':<10}{'Status':<10}{'Action':<10}{'Env After'}")
        for step in range(1, steps + 1):
            percept = self.perceive()
            action = self.decide_action(percept)
            self.act(action)
            print(f"{step:<6}{percept[0]:<10}{percept[1]:<10}{action:<10}{self.environment.display()}")
        print(f"\nTotal cleaning actions performed (performance score): {self.performance}")


if __name__ == "__main__":
    env = SimpleEnvironment()
    agent = ReflexVacuumAgent(env)
    print("Initial environment state:", env.display())
    print()
    agent.run(steps=6)
```

Output:

```
Initial environment state: {'A': 'Dirty', 'B': 'Dirty'}

Step  Location  Status    Action    Env After
1     A         Dirty     Clean     {'A': 'Clean', 'B': 'Dirty'}
2     A         Clean     Right     {'A': 'Clean', 'B': 'Dirty'}
3     B         Dirty     Clean     {'A': 'Clean', 'B': 'Clean'}
4     B         Clean     Left      {'A': 'Clean', 'B': 'Clean'}
5     A         Clean     Right     {'A': 'Clean', 'B': 'Clean'}
6     B         Clean     Left      {'A': 'Clean', 'B': 'Clean'}

Total cleaning actions performed (performance score): 2
```

Traveling Salesman Problem (TSP)

Aim:

To find a route that visits every city exactly once and returns to the start, minimizing total travel distance — using both a fast heuristic (Nearest Neighbor) and an exact method (Brute Force) for comparison on a small instance.

Theory:
TSP is an NP-hard combinatorial optimization problem: with n cities there are (n-1)!/2 distinct tours, making exhaustive search infeasible beyond small n. Two approaches are shown:

- **Nearest Neighbor (greedy heuristic):** starting from a city, repeatedly move to the closest unvisited city until all are visited, then return home. It runs in $O(n^2)$ time but can miss the optimal tour because early "greedy" choices may force long detours later.
- **Brute Force (exact):** evaluate every permutation of the remaining cities (fixing one start city to avoid duplicate rotations) and keep the shortest. Guarantees optimality but runs in O((n-1)!), so it only scales to a handful of cities.

Comparing the two illustrates the classic speed vs. optimality trade-off common in AI search and optimization.

Code:

```
"""

Traveling Salesman Problem - Nearest Neighbor Heuristic + Brute Force check (small instance)

"""

import math

import itertools



def calculate_distance(city1, city2):

    return math.sqrt((city1[0] - city2[0]) ** 2 + (city1[1] - city2[1]) ** 2)



def nearest_neighbor_tsp(cities):

    num_cities = len(cities)

    if num_cities < 2:

        return [0], 0
```

```
    start_city_index = 0
    unvisited = set(range(num_cities))
    unvisited.remove(start_city_index)
    current = start_city_index
    tour = [start_city_index]
    total_distance = 0

    while unvisited:
        nearest_index = -1
        min_distance = float('inf')
        for candidate in unvisited:
            d = calculate_distance(cities[current], cities[candidate])
            if d < min_distance:
                min_distance = d
                nearest_index = candidate
        tour.append(nearest_index)
        unvisited.remove(nearest_index)
        total_distance += min_distance
        current = nearest_index

    total_distance += calculate_distance(cities[current], cities[start_city_index])
    tour.append(start_city_index)
    return tour, total_distance


def brute_force_tsp(cities):
    """Exact solution by checking all permutations - only feasible for small n."""
    num_cities = len(cities)
    indices = list(range(1, num_cities))  # fix city 0 as start
```

```
    best_tour = None
    best_distance = float('inf')

    for perm in itertools.permutations(indices):
        tour = [0] + list(perm) + [0]
        distance = sum(
            calculate_distance(cities[tour[i]], cities[tour[i + 1]])
            for i in range(len(tour) - 1)
        )
        if distance < best_distance:
            best_distance = distance
            best_tour = tour

    return best_tour, best_distance


if __name__ == "__main__":
    cities = [(0, 0), (1, 5), (3, 2), (7, 8), (4, 4)]
    city_labels = ['A', 'B', 'C', 'D', 'E']

    nn_tour, nn_distance = nearest_neighbor_tsp(cities)
    bf_tour, bf_distance = brute_force_tsp(cities)

    print("Cities (coordinates):")
    for label, coord in zip(city_labels, cities):
        print(f"  {label}: {coord}")

    print("\n--- Nearest Neighbor Heuristic ---")
    print("Tour:", " -> ".join(city_labels[i] for i in nn_tour))
    print(f"Total distance: {nn_distance:.2f}")
```

```
print("\n--- Brute Force (Optimal) ---")
print("Tour:", " -> ".join(city_labels[i] for i in bf_tour))
print(f"Total distance: {bf_distance:.2f}")
```

Output:

```
Cities (coordinates):
  A: (0, 0)
  B: (1, 5)
  C: (3, 2)
  D: (7, 8)
  E: (4, 4)

--- Nearest Neighbor Heuristic ---
Tour: A -> C -> E -> B -> D -> A
Total distance: 26.34

--- Brute Force (Optimal) ---
Tour: A -> B -> D -> E -> C -> A
Total distance: 22.65
```